



Plan de pruebas de la aplicación Bicis

Historia de usuario: Ver Información Detallada de Red de Bicicletas

Historia de usuario. Yo, como usuario, quiero poder obtener toda la información posible de una red de bicicletas, de manera que pueda determinar si utilizaré esa red en función de detalles no visibles a nivel global, pero que pueden ser relevantes para mí, como la oferta de bicis eléctricas o el número y/o nombre de las empresas que soportan la red.

Interfaz esperada: mock-ups del Sprint 0 (Docs/Mockups/Mock-ups_Sprint0_Pruebas.pdf), que muestran la pantalla "Detalles de la red" con nombre, ciudad y país, Latitud, Longitud, Bicis eléctricas, N° de empresas y la lista de Empresas operadoras; un guion para los datos no disponibles; y el mensaje "No hay información disponible para esta red" cuando no hay detalle.

Los niveles de prueba que se van a aplicar son los siguientes:

- Pruebas de aceptación. Se definirán siguiendo una estrategia basada en la historia de usuario y sus cuatro pruebas (éxito, datos ausentes, datos anómalos y red sin información detallada), y se ejecutarán de forma manual sobre la versión debug de la aplicación, que carga los datos desde ficheros JSON locales (JsonNetworksRepository).
- Pruebas de integración. La estrategia para la definición del orden de las pruebas de integración será jerárquica (ascendente). Se probará:
 - La integración entre el presenter de detalle y las clases de dominio (Network, Location). Se utilizarán técnicas de prueba de métodos y caja negra, con JUnit y una vista simulada con Mockito.
 - La integración entre las tres capas: DetailsView, presenter de detalle y repositorio (JsonNetworksRepository con los datos del Anexo 1). Prueba funcional completa definida con la técnica de casos de uso y automatizada con Espresso y Hilt.
- Pruebas unitarias. Se aplicarán a la lógica nueva de la capa de presentación: cálculo del número de empresas, validación de coordenadas, conversión de booleanos a Sí/No y detección de red sin información. Se usará prueba de métodos con caja negra (partición equivalente y AVL), con JUnit y Mockito (y Robolectric cuando haga falta un Context, como en ExampleTest).

La aplicación sigue el patrón Modelo-Vista-Presentador (MainView, MainPresenter, IMainContract). Actualmente la vista de detalle (DetailsView) no tiene presenter, por lo que los nombres DetailsPresenter, IDetailsContract y los de sus métodos que aparecen en este plan son una propuesta siguiendo ese mismo patrón y deberán ajustarse a los de la implementación final.

A continuación, se muestra una especificación detallada de los casos de prueba a aplicar en cada nivel mencionado anteriormente.

PRUEBAS DE ACEPTACIÓN

En base a la historia de usuario se identifican los siguientes escenarios:

A1. HU: Ver Información Detallada de Red de Bicicletas



- a. Red con toda la información disponible (caso de éxito, Prueba 1)
- b. Red con datos ausentes (Prueba 2)
- c. Red con datos anómalos (Prueba 3)
- d. Red sin información detallada (Prueba 4)

Procedimiento común a todos los escenarios: (1) el usuario abre la aplicación y espera a que se cargue la lista principal de redes; (2) selecciona la red indicada en la columna Entrada de la Tabla 2; (3) la aplicación abre la vista "Detalles de la red"; (4) se comprueba que lo mostrado coincide con la columna Resultado. En los escenarios a, b y c el número de empresas se calcula a partir de la lista company. Los datos concretos de cada red están en el Anexo 1 y los resultados esperados en la Tabla 2, que se usa tanto para la ejecución manual como para la automatizada.

Criterios comunes de verificación:

- Los valores booleanos (ebikes) se muestran como "Sí" o "No", nunca como true/false.
- Un dato ausente o con valor anómalo se muestra como un guion ("-"). La ausencia de ebikes no significa que la red no tenga bicicletas eléctricas: se muestra "-". Solo el valor false se muestra como "No".
- El N° de empresas coincide con el número de elementos de company y solo se muestra si company está disponible y es un array de textos no vacíos; en otro caso se muestra "-".
- Las coordenadas son válidas si la latitud está en [-90, 90] y la longitud en [-180, 180], ambos extremos incluidos. Se muestran con 4 decimales, como en el mock-up.

PRUEBAS UNITARIAS

Se aplican técnicas de clases de equivalencia y AVL para la definición de los casos de prueba de cada método. Además, se deberá garantizar la cobertura de sentencias y de condición/decisión. Las entradas se expresan como el valor del campo en el JSON del servicio; "ausente" indica que la clave no aparece.

Pruebas unitarias de la lógica de presentación

Se aplica prueba de métodos, siendo los casos de prueba definidos para cada método los que se exponen a continuación.

- Método numEmpresas(company)

Tabla 1. Casos de prueba del cálculo del número de empresas

Identificador	Entrada	Valor esperado
UDP.1a	["Barcelona de Serveis Municipals, S.A. (BSM)", "CESPA", "PBSC"] (lista válida, varios elementos)	3
UDP.1b	["UTE BICIS SANTANDER"] (lista válida, un elemento, AVL)	1
UDP.1c	ausente (company no viene en el JSON)	null (no se calcula; se muestra "-")
UDP.1d	[] (lista vacía, AVL)	null (se trata como dato no disponible)
UDP.1e	["", "CESPA"] (elemento vacío)	null (dato anómalo)
UDP.1f	[" "] (elemento solo con espacios)	null (dato anómalo)
UDP.1g	[123, "CESPA"] (elemento no textual)	null (dato anómalo)
UDP.1h	"CESPA" (texto, no es una lista)	null (dato anómalo)



- Método `formateaBooleano(ebikes)`

Identificador	Entrada	Valor esperado
UDP.2a	true	"Sí"
UDP.2b	false	"No"
UDP.2c	ausente	"-" (dato no disponible, no "No")
UDP.2d	"si" (texto)	"-" (dato anómalo)
UDP.2e	1 (número)	"-" (dato anómalo)

- Método `formateaLatitud(latitude)`

Identificador	Entrada	Valor esperado
UDP.3a	41.3850639 (valor interior)	"41.3851"
UDP.3b	-90 (límite inferior, AVL)	"-90.0000"
UDP.3c	-90.0001 (justo por debajo, AVL)	"_"
UDP.3d	90 (límite superior, AVL)	"90.0000"
UDP.3e	90.0001 (justo por encima, AVL)	"_"
UDP.3f	0	"0.0000"
UDP.3g	ausente	"_"
UDP.3h	"abc" (no numérico)	"_"

- Método `formateaLongitud(longitude)`

Identificador	Entrada	Valor esperado
UDP.4a	-3.7038 (valor interior)	"-3.7038"
UDP.4b	-180 (límite inferior, AVL)	"-180.0000"
UDP.4c	-180.0001 (justo por debajo, AVL)	"_"
UDP.4d	180 (límite superior, AVL)	"180.0000"
UDP.4e	180.0001 (justo por encima, AVL)	"_"
UDP.4f	ausente	"_"
UDP.4g	"abc" (no numérico)	"_"

- Método `tieneInformacionDetallada(network)`

Identificador	Entrada	Valor esperado
UDP.5a	Red Bicing (todos los datos)	true
UDP.5b	Red Bici Sur (solo nombre, ciudad y país)	false
UDP.5c	Red con solo ebikes: false	true (false es un dato)
UDP.5d	Red con solo latitude	true

Pruebas unitarias del presenter de detalle

Se prueba `DetailsPresenter` aislado, con una vista simulada (mock de `IDetailsContract.View`) y verificando con Mockito las llamadas que hace a la vista.

- Método `init(view)` con la red recibida de la lista principal

Identificador	Entrada	Valor esperado
UDP.6a	Red Bicing	Se llama a <code>showDetails</code> con lat "41.3851", lon "2.1734", ebikes "Sí", nº empresas "3" y las 3



		empresas; no se llama a showNoDetailsMessage
UDP.6b	Red Bici Norte (datos ausentes)	Se llama a showDetails con lat "43.2630" y "-" en longitud, ebikes, nº empresas y empresas
UDP.6c	Red Bici Sur (sin información)	Se llama a showNoDetailsMessage; no se llama a showDetails
UDP.6d	null (no llega red en el Intent)	Se llama a showNoDetailsMessage; no se produce excepción

PRUEBAS DE INTEGRACIÓN

El orden de las pruebas y los casos de prueba a realizar serían los siguientes:

1. DetailsPresenter con Network y Location.

Los casos de prueba son los mismos definidos como UDP.6x en la sección de pruebas unitarias, aquí renombrados como IDP.6x, pero usando objetos Network reales obtenidos al deserializar con Gson los JSON del Anexo 1 (en lugar de construirlos a mano). Así se comprueba también que el modelo conserva la diferencia entre dato ausente y dato con valor false.

2. DetailsView con DetailsPresenter y JsonNetworksRepository.

Los casos de prueba se definen en base a los escenarios identificados en las pruebas de aceptación, aunque en este caso se automatizarían con Espresso (partiendo de ExampleUITest) y deben partir de un estado conocido de los datos. En este caso, el estado inicial es el mostrado en el Anexo 1 y en base a ello, los casos de prueba que se definen son los que se muestran en la Tabla 2. Al no modificar datos, los casos pueden ejecutarse en cualquier orden.

Tabla 2. Casos de prueba de integración funcional completa

Identificador	Entrada	Resultado
A1.a.1	Seleccionar "Bicing" (Barcelona)	Cabecera: "Bicing", "Barcelona, ES" Latitud: "41.3851" Longitud: "2.1734" Bicis eléctricas: "Sí" Nº de empresas: "3" Empresas: "Barcelona de Serveis Municipals, S.A. (BSM)", "CESPA", "PBSC"
A1.a.2	Seleccionar "Bici Oeste" (Oviedo)	Latitud: "43.3614" Longitud: "-5.8494" Bicis eléctricas: "No" Nº de empresas: "2" Empresas: "Empresa A", "Empresa B"
A1.b.1	Seleccionar "BiciMAD" (Madrid) (sin ebikes)	Latitud: "40.4168" Longitud: "-3.7038" Bicis eléctricas: "-" (no "No") Nº de empresas: "1" Empresas: "Empresa Municipal de Transportes de Madrid, S.A."
A1.b.2	Seleccionar "TUEBICI" (Santander) (sin ebikes)	Latitud: "43.4623" Longitud: "-3.8100" Bicis eléctricas: "-"



		Nº de empresas: "1" Empresas: "UTE BICIS SANTANDER"
A1.b.3	Seleccionar "Bici Norte" (Bilbao) (sin longitude, ebikes ni company)	Latitud: "43.2630" Longitud: "-" Bicis eléctricas: "-" Nº de empresas: "-" Empresas: "-"
A1.c.1	Seleccionar "Bici Coordinadas" (Gijón) (latitud 95.5, longitud -181.2)	Latitud: "-" Longitud: "-" Bicis eléctricas: "Sí" Nº de empresas: "1" Empresas: "Empresa A"
A1.c.2	Seleccionar "Bici Límites" (Ushuaia) (latitud -90, longitud 180)	Latitud: "-90.0000" Longitud: "180.0000" (valores límite válidos, se muestran)
A1.c.3	Seleccionar "Bici Anómala" (León) (ebikes "si", company ["", "Empresa B"])	Latitud: "42.5987" Longitud: "-5.5671" Bicis eléctricas: "-" Nº de empresas: "-" Empresas: "-"
A1.c.4	Seleccionar "Bici Texto" (Burgos) (company "Empresa C" como texto, ebikes 1)	Bicis eléctricas: "-" Nº de empresas: "-" Empresas: "-" La aplicación no se cierra
A1.d.1	Seleccionar "Bici Sur" (Cádiz)	Cabecera: "Bici Sur", "Cádiz, ES" Mensaje: "No hay información detallada disponible para esta red" No se muestran los campos Latitud, Longitud, etc.

ANEXO 1. DATOS DE PRUEBA

Los datos con los que se lanza la aplicación en modo debug, que es necesario conocer para la realización de las pruebas de integración y aceptación, son los de la Tabla 3. Las tres primeras redes ya existen en `res/raw/networks.json`; el resto hay que añadirlas como datos de prueba. Para no alterar la lista que ve el equipo en debug, se recomienda cargarlas desde un JSON propio de las pruebas, sustituyendo el `RepositoriesModule` con `@TestInstallIn`.

Tabla 3. Estado inicial de los datos de prueba

id / nombre	Ciudad, país	latitude	longitude	ebikes	company	Origen
bicing / Bicing	Barcelona, ES	41.3850639	2.1734035	true	["Barcelona de Serveis Municipals, S.A. (BSM)", "CESPA", "PBSC"]	Existente
bicimad / BiciMAD	Madrid, ES	40.4168	-3.7038	ausente	["Empresa Municipal de Transportes de Madrid, S.A."]	Existente
tuebici / TUEBICI	Santander, ES	43.4623057	-3.8099803	ausente	["UTE BICIS SANTANDER"]	Existente
bicioeste / Bici Oeste	Oviedo, ES	43.3614	-5.8494	false	["Empresa A", "Empresa B"]	Nuevo



bicinorte / Bici Norte	Bilbao, ES	43.2630	ausente	ausente	ausente	Nuevo
bicicoord / Bici Coordenadas	Gijón, ES	95.5	-181.2	true	["Empresa A"]	Nuevo
bicilimites / Bici Límites	Ushuaia, AR	-90	180	true	["Empresa A"]	Nuevo
bicianomala / Bici Anómala	León, ES	42.5987	-5.5671	"si"	["", "Empresa B"]	Nuevo
bicitexto / Bici Texto	Burgos, ES	42.3439	-3.6969	1	"Empresa C"	Nuevo
bicisur / Bici Sur	Cádiz, ES	ausente	ausente	ausente	ausente	Nuevo

Observaciones sobre el código actual

Revisando el código de la aplicación se detectan los siguientes puntos que afectan a estas pruebas y que conviene resolver al implementar la historia:

- `DetailsView` solo muestra el nombre y la ciudad (`tvName`, `tvCity`). La historia todavía no está implementada, así que hoy fallarían todos los casos de la Tabla 2.
- `Location` solo mapea `city`. Faltan `latitude`, `longitude` y `country`, que conviene declarar como `Double/String` para poder distinguir un dato ausente (`null`).
- `Network.ebikes` es un `boolean` primitivo: si el JSON no trae ebikes, Gson deja `false` y se mostraría "No" en vez de "-" (fallaría A1.b.1 y A1.b.2, que usan datos reales). Debe pasar a `Boolean`.
- Gson convierte el texto "si" en `false` sin avisar, y un número en `ebikes` o un texto en `company` (que espera un array) lanza `JsonSyntaxException`. Para detectar valores anómalos (A1.c.3, A1.c.4) hay que leer esos campos sin tipar (`JsonElement`) o con un `TypeAdapter` propio.
- `Utils.parseNetworks` y `Utils.parseNetwork` solo capturan `IOException`. Una `JsonSyntaxException` por un dato anómalo de una sola red haría fallar la carga de toda la lista y cerraría la aplicación.
- El servicio también devuelve `gbfs_href`, `system` y `source`, que no aparecen en el mock-up. Si se decide mostrarlos para cumplir con "toda la información posible", habría que añadir casos para ellos.

Decisiones tomadas por defecto en este plan, a confirmar con el responsable de producto: una lista `company` vacía, o con algún elemento vacío o no textual, se trata entera como dato anómalo ("-"); las coordenadas se muestran con 4 decimales y el país con el código que da el servicio ("ES"); y el texto exacto del mensaje del escenario d es "No hay información detallada disponible para esta red".